

경력기술서

지원자: 박정환 | Backend Developer · Full-Stack | 총 경력 4년

경력 요약

기간	회사명	직위	담당 업무	고용형태
2020.11 - 2024.10 4년	(주)가비아	대리 · 파트장	전자결재 시스템 백엔드 개발·운영 총괄 (PHP 레거시 → Java REST API 차세대 전환)	정규직
2024.10 - 2026.03 1년 6개월	개인 프로젝트	단독 개발	로스트아크 인벤 사건사고 검색기 — 폴스택 + AI OCR 마이크로서비스 설계·구현	개인

(주)가비아

2020.11 - 2024.10 · 4년 · 정규직 대리

업종 IT 서비스 / SaaS	직위 대리 · 파트장	담당 전자결재 백엔드	팀 규모 전자결재 파트
---------------------	----------------	----------------	-----------------

전자결재 차세대 REST API 개발

2022.08 - 2024.10 · 2년 2개월

역할: 파트장 · 백엔드 설계·구현 총괄 (단독 주도)

Java 17

Spring Boot 3

MySQL / Aurora

MongoDB

Kafka

Docker

Kubernetes

JUnit5

Mockito

ERP 연동

그룹웨어 연동

프로젝트 개요

10년 이상 운영된 PHP 모놀리식 전자결재 시스템을 Java/Spring Boot REST API 기반 차세대 구조로 전환하는 프로젝트. 기능 확장성·성능·운영 안정성을 목표로 MSA 방향의 아키텍처를 직접 설계하고 구현했습니다.

주요 수행 내용

- REST API 설계·구현 주도:** 전자결재 핵심 도메인(문서·결재선·상태·이력·권한) API 20개 이상 재설계 및 구현. 외부 ERP·그룹웨어·글로벌 시스템 연동 규격 표준화.
- 멀티 DB 아키텍처 설계:** 정형 결재 데이터는 MySQL/Aurora, 문서 양식·첨부 등 비정형 데이터는 MongoDB로 분리. 도메인 특성에 맞는 저장소 목적 분리 설계.
- Kafka 기반 이벤트 자동 복구 구조:** 모든 처리 이벤트를 Kafka에 기록하고 네트워크 정상화 시 배치가 자동 재처리하는 멍등성 보장 구조 설계·구현. 장애 시에도 데이터 유실 없는 안정적 운영 확보.
- 트랜잭션 범위 재설계:** 로그 분석으로 불필요하게 넓은 트랜잭션이 DB 락을 과도하게 유지함을 확인. 데이터 정합성이 실제로 필요한 구간에만 트랜잭션을 적용하도록 재설계. 성능 테스트 직접 주도하여 검증.

- ▶ **인증·인가 체계 중앙화:** 역할·문서 상태·조직 관계에 따라 행위별 권한이 복잡하게 달라지는 도메인 특성 분석. 인증은 공통 필터, 인가는 도메인 서비스 레벨에서 중앙화하여 신규 API 추가 시 중복 구현 불필요한 구조 확보.
- ▶ **Docker · Kubernetes 배포 파이프라인 구축:** 전자결재 서비스 컨테이너화 및 Kubernetes 롤링 배포 파이프라인 구축. 서비스 단위 독립 배포 및 무중단 배포 체계 마련.
- ▶ **JUnit5 기반 테스트 체계 도입:** 레거시에 부재했던 단위·통합 테스트 설계·도입. API 리팩토링 시 회귀 버그 방지 체계 확립.
- ▶ 레거시 PHP 모놀리식 코드 분석 및 비즈니스 로직 분리·점진적 리팩토링 주도.

주요 성과

3,000ms → 100ms

트랜잭션 범위 재설계
평균 API 응답 시간 97% 개선

월 수십 건 → 0건

Kafka 자동 복구 도입
수동 운영 개입 완전 제거

MSA 기반 구축

단일 레거시 → 서비스 단위
독립 배포 체계 마련

전자결재 유지보수 / 고도화

2020.11 – 2024.10 · 4년

역할: 파트장 · 레거시 백엔드 운영 및 성능 개선 담당

PHP

CodeIgniter

Laravel

MySQL / Aurora

Linux

DB Partitioning

Index Tuning

프로젝트 개요

사내 전자결재 레거시 서비스 백엔드 개발·운영 담당. PHP 모놀리식 구조를 안정적으로 운영하면서 기능 고도화·대용량 데이터 성능 개선을 지속하고, 차세대 REST API 전환을 위한 구조적 기반 마련.

주요 수행 내용

- ▶ **대용량 문서 검색 성능 개선:** 전자결재 문서 1,000만 건+ 누적 환경에서 슬로우 쿼리 로그 분석 → Full Table Scan·비효율 서브쿼리·인덱스 미활용 구간 특정. office_no 기반 파티셔닝·샤딩 적용, 복합 인덱스 재설계, 불필요 조인 제거 후 단계적 검증.
- ▶ **레거시 안정 운영:** 전자결재 핵심 기능(문서·결재선·상태·이력) 추가·보완 및 반복 장애 원인 분석·제거.
- ▶ **차세대 전환 기반 마련:** 10년 이상 축적된 전자결재 비즈니스 로직·예외 케이스·운영 노하우 체계적 문서화. 신규 Java REST API 서버 도입을 위한 연계 구조 설계.
- ▶ 레거시·차세대 병행 운영 기간 동안 데이터 정합성 유지 구조 설계.

주요 성과

1,500ms → 300ms

DB 파티셔닝·인덱스 최적화로
대용량 문서 검색 80% 개선

1,000만 건+

대용량 전자결재 문서 환경
안정적 조회 체계 확립

개인 프로젝트 퇴직 후 자기 주도 개발

2024.10 – 2026.03 · 1년 6개월

로스트아크 인벤 사건사고 검색기

2024.10 – 2026.03

역할: 전체 설계 및 구현 단독 (풀스택 · AI OCR 서비스)

Java 17

Spring Boot 3.2

CompletableFuture

Jsoup

JUnit5

WireMock

React

TypeScript

Python

FastAPI

PaddleOCR

OpenCV

프로젝트 개요

게임(로스트아크) 파티 구성 시 악성 유저를 자동으로 걸러내는 풀스택 웹 서비스. React 프론트엔드 · Spring Boot 병렬 처리 백엔드 · Python FastAPI AI OCR 마이크로서비스 3개 서비스 레이어를 처음부터 단독 설계·구현.

SPRING BOOT 백엔드 — 병렬 처리 최적화

- ▶ **병렬 처리 구조 설계:** 캐릭터 6개 × 15페이지 × 본문 확인 = 수백 건 외부 HTTP 요청 문제 분석. sterm 파라미터가 +10,000 단위로 규칙적으로 증가함을 파악 → 나머지 14페이지 URL 선계산 후 CompletableFuture 동시 요청으로 전환.
- ▶ 목적별 ThreadPool 3개 분리 운영: 페이지 fetch 전용, 게시글 본문 확인 전용(×10), 부캐 검색 전용(×6) — 전체 강화 검색 수 분 → 수 초.
- ▶ **크롤링 신뢰도 확보:** stype=content 검색의 오타 문제를 개별 게시글 fetch 후 "대상자:" 섹션 내 닉네임 존재 여부 정밀 검증으로 해결. 실사용자 제보 기반 5가지 분리자 패턴 오타 방지 로직 구현.
- ▶ Lost Ark Open API 연동으로 부캐 자동 조회 — 닉네임 → 캐릭터 6개 병렬 검색.
- ▶ WireMock 기반 통합 테스트 — 외부 의존성 없이 크롤링·파싱 로직 검증.

PYTHON AI OCR 마이크로서비스

- ▶ **OCR 엔진 선택:** Tesseract.js(브라우저, 한국 게임 폰트 인식 부족) → EasyOCR(범용 모델 한계) 시도 후 PaddleOCR PP-OCRv4 한국어 특화 모델 채택. Windows oneDNN 충돌로 버전 2.7.x 고정.
- ▶ **전처리 파이프라인 설계:** HSV 마스크로 흰색 픽셀 추출·반전 처리 → 한국 게임 UI 특성에 맞는 인식을 확보.
- ▶ **위치 기반 3단계 닉네임 추출 전략:** ①Lv.XX 오른쪽 탐색 → ②서버명 아래 탐색 → ③행 클러스터 탐색 (최후 폴백) 순차 적용.
- ▶ **자모 혼동 오인식 보정:** | ↔ ㅌ 등 OCR 오인식 패턴 분석 후 후보 닉네임 생성 로직 구현.

REACT 프론트엔드

- ▶ **Document Picture-in-Picture API 활용:** 일반 팝업은 게임 전체화면에 가려지는 문제 → Chrome 116+ Document PiP API로 항상 최상위 플로팅 창 구현.
- ▶ **화면 공유 OCR 파이프라인:** getDisplayMedia API로 화면 캡처 → Canvas ROI 추출(% 좌표, 해상도 무관) → FastAPI 전송.
- ▶ canvas 조건부 렌더링 시 ref null 버그 → display:none/grid로 가시성만 제어하는 방식으로 해결.